

Identificación del problema y análisis de requerimientos

Cliente	Los estudiantes de la Facultad de Negocios y Economía de la Universidad Icesi
Usuario	Los estudiantes de Economía y Negocios Internacionales.
Requerimientos funcionales	● RF1 Realizar varios juegos en una misma ejecución del programa. ● RF2 El programa debe mostrar el tablero de juego por medio de una matriz. ● RF3 Ofrecer el modo de juego estándar. ● RF4 Ofrecer el modo de juego personalizado. ● RF5 Registrar al jugador. ● RF6 Registrar ataques. ● RF7:Llevar la cuenta de quien gana cuantas veces. ● RF8 Registrar los barcos
Contexto del problema	Los estudiantes de la Facultad de Negocios y Economía de la Universidad Icesi, necesitan un programa para simular el juego batalla naval, el cual se debe ejecutar desde la terminal, el programa simula un juego de combate naval donde 0 simboliza agua, 1 simboliza que está ocupado y 3 está hundido. Todo esto por medio de matrices y menús.
Requerimientos no funcionales	R1 El sistema debe mostrar información de los menús, el estado de juego y mensajes de error de manera clara y organizada a través de la consola. R2 El sistema debe ofrecer una experiencia de usuario fluida a partir de un bajo tiempo de respuesta. R3 El programa debe permitir realizar varios juegos en una misma ejecución del programa y llevar la cuenta de quien gana cuantas veces
Requerimientos de proceso	El programa se debe entregar el sábado 5 de abril del 2025 antes de las 10 de la noche.

Requerimiento Funcional	Nombre de la Clase	Nombre del método
Req Func001 RF1 Realizar varios juegos en una misma ejecución del programa.	Executable	+menu(): void +selectTypeOfGame() : void
	Controller	+playStandartGame() : String +playCostumeGame(shipNum : int) : String
Req Func002 RF2 El programa debe mostrar el tablero de juego por medio de una matriz.	Executable	+createBoard(): void +showBoard(): void
	Controller	+createBoard(userName : String) : String -searchBoard(userName : String) : Board +showBoard(username: String) : String

	User	+createBoard(): String
	Board	+Board()
Req Func003 Ofrecer el modo de juego estándar.	Executable	+playGame() : void +createStandarShips() : void +showBoard() : void +humanAttack() : void +machineAttack(): void +showWinCounts() : void
	Controller	+typeGame(typeOfPlay : int) : String +playStandartGame() : String -searchUser(name : String) : User

		+convertCoordinate(xCoordinate : int, yCoordinate : int) : Coordinate +convertOrientation(orientation : int) : Orientation +createShip(name : String, coordinate : Coordinate int, orientation : Orientation, length : int) : boolean +registerShipsRandom() : boolean +addMachineShip() : boolean +humanAttack(xCoordinate : int, yCoordinate : int) : boolean +machineAttack() : String -searchBoard(username : String) : Board +showBoard(username : String) : String +showMachineBoard() : String +showWinCounts() : String
	User	+createBoard() : String +newStandarShip() : boolean +addShip(ship : Ship) : boolean

		<code>+searchBoard() : Board</code> <code>+attack(coordinate : Coordinate) : String</code> <code>+getWinCount() : int</code> <code>+setWinCount(winCount : int) : String</code>
	Board	<code>+Board()</code> <code>+standarShip() : Ship</code> <code>+addShip(shipType : int, x : int, y : int) : boolean</code> <code>+attack(xCoordinate : int, yCoordinate) : boolean</code> <code>+showBoard() : String</code> <code>+lifesShips() : boolean</code>
	Ship	<code>+Ship(name : String, lenght : int, orientation : Orientation)</code> <code>+getShipOrientation() : Orientation</code> <code>+setShipOrientation(shipOrienta tion : Orientation) : boolean</code> <code>+markHit() : void</code> <code>+getLife() : ShipSunk[]</code> <code>+setLife(life : ShipSunk[])</code>

Req Func004 Ofrecer el modo de juego personalizado.	Executable	+playGame() : void +createShip() : void +createMachineShip() : void +placePlayerShips() : void +placeMachineShips() : void
	Controller	+playCostumeGame(shipNum : int) : String +convertCoordinate(xCoordinate : int, yCoordinate : int) : Coordinate +convertOrientation(orientation : int) Orientation +createShip(name : String, coordinate : Coordinate int, orientation : Orientation, length : int) : boolean
	User	+createBoard() : String +addShip(ship : Ship):boolean

	Board	+Board() +addShip(shipType : int, x : int, y : int) : boolean
	Ship	+Ship(name : String, lenght : int, orientation : Orientation)
Req Func005 Registrar al jugador.	Executable	+registerPlayer() : void
	Controller	createUser(name : string) : String
	User	+User(name : String)
Req Func006 Registrar los ataques.	Executable	+humanAttack():void +machineAttack() : void
	Controller	+humanAttack(xCoordinate : int, yCoordinate : int) : boolean +machineAttack() : String +someoneWin(Board) : User

	User	+attack(coordinate : model.Model.Coordinate) : String
	Board	+attack(xCoordinate : int, yCoordinate) : boolean +lifesShips() : boolean
	Ship	+markHit() : void +getLife() : ShipSunk [] +setLife(life : ShipSunk[]): boolean
	Coordinate	+getXCoordinate() : int +getYCoordinate() : int
Req Func007 Llevar la cuenta de quien gana cuantas veces.	Executable	+humanAttack():void +machineAttack() : void
	Controller	+someoneWin(Board) : User +showWinCounts() : String
	User	+getWinCount() : int

		+setWinCount(winCount : int) : String +restartBoard() : boolean
	Board	+livesShips() : boolean
	Ship	+getLife() : ShipSunk[] +setLife(life : ShipSunk[]) : boolean
Req Func008 Registrar los barcos	Executable	+createShip():void
	Controller	+createShip(name : String, orientation : Orientation, length : int) : boolean +startCoordinates(x : int, y : int) : Coordinate
	User	+addShip(ship : Ship) : boolean +createCoordinate(coordinate : Coordinate) : boolean
	Board	+addShip(shipType : int, x : int, y : int) : boolean +addCoordinate(coordinate : model.Model.Coordinate) : void

	Ship	+Ship(name : String, lenght : int, orientation : Orientation) +addStartCoordinate(x : int, y : int) : String
--	------	--